

DEVELOPER HANDOVER GUIDE

Kobuki-Based Dual-Arm Mobile Manipulator

ROS 2 Jazzy | Gazebo Harmonic | bimanual_ws

Complete file-by-file reference and code-backed walkthrough

Who this document is for

You know ROS 2 and Gazebo basics but have never seen this project. This guide assumes nothing about the project itself. It describes every file, what each one contains, how they connect, every topic and parameter, and then walks through the entire simulation run step by step - each step tied back to the exact file and function that causes it. Read section 11 if you only read one thing: it is the run itself, narrated against the code.

How to read this

Sections 1-3	Orientation: what the robot does, how to build and run it, and the mental model.
Section 4	Repository map - every file with a one-line purpose.
Sections 5-7	File-by-file deep dive, one package at a time.
Section 8	How the files interact - the data flow.
Sections 9-10	Complete interface and geometry reference tables.
Section 11	The full runtime walkthrough, step by step, with code citations. The core of this document.
Sections 12-15	Tuning cookbook, tests, gotchas, glossary.

One-line summary of the system

A two-armed robot on a round mobile base spins to survey a room, spots colour-coded objects lying on their side, drives up to one, grips it with both hands, rotates it upright in mid-air, drives to a shelf that it finds with its LiDAR, and sets the object down standing - then repeats for the next object.

1. What the System Does

The robot performs a full fetch-and-right cycle autonomously. Objects start lying on their side on the floor. The robot must find them, pick them up with two arms, turn them upright while holding them in the air, and place them standing on a shelf. The bimanual arrangement exists for exactly that mid-air rotation: each arm has 4 degrees of freedom and cannot rotate a grasped object about a horizontal axis on its own, but two arms holding opposite ends can rotate it together like a steering wheel.

The seven phases

#	Phase	Meaning
1	SEARCH	Spin in place and record every object seen
2	APPROACH	Drive toward the chosen object
3	ALIGN	Centre the object before the final blind approach
4	IK GRASP	Both grippers descend and close on it
5	CO-MANIPULATION	Object is rigidly bound to the gripper; both arms move as a unit
6	REORIENTATION	Lift and rotate 90 degrees to upright
7	PLACEMENT	Dock at the shelf, lower, release, retreat

The three objects

Object	Geometry	Mass	Colour	Shelf height at release
bottle	60 mm dia. x 200 mm cylinder	150 g	orange	0.235 m (object centre)
bowl	80 mm dia. x 100 mm cup	200 g	blue	0.185 m
cracker_box	60 x 80 x 180 mm box	250 g	green	0.225 m

Colour matters: it is the only thing that distinguishes objects to the perception system. Everything else in the scene is deliberately desaturated so it cannot be mistaken for an object.

2. Build and Run

Requirements

Pop!_OS / Ubuntu 24.04 (Noble) - required for ROS 2 Jazzy packages

```
sudo apt install -y \
  ros-jazzy-desktop ros-jazzy-ros-gz ros-jazzy-gz-ros2-control \
  ros-jazzy-ros2-controllers ros-jazzy-xacro \
  python3-opencv python3-numpy python3-colcon-common-extensions
```

Build

```
cd ~/kuboki/PROJECT/DEV/bimanual_ws      # wherever the workspace is
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

Run

```
# main world: three fallen objects handled one after another
ros2 launch kobuki_bimanual_gazebo sim.launch.py

# single-object worlds
ros2 launch kobuki_bimanual_gazebo sim.launch.py world:=bottle
ros2 launch kobuki_bimanual_gazebo sim.launch.py world:=bowl
ros2 launch kobuki_bimanual_gazebo sim.launch.py world:=box

# load the sim but do NOT start the mission (drive it manually)
ros2 launch kobuki_bimanual_gazebo sim.launch.py autostart:=false
```

If the GUI misbehaves on your machine

Prefix the launch with QT_QPA_PLATFORM=xcb (this is what has been used during development). On virtual machines without GPU passthrough, export LIBGL_ALWAYS_SOFTWARE=1 as well.

Useful inspection commands while it runs

```
ros2 topic echo /detections      # what perception sees, JSON
ros2 topic hz /camera /scan      # sensors alive?
ros2 control list_controllers    # all four should be active
ros2 topic echo /joint_states    # 12 joints
gz topic -l                     # raw Gazebo topic list
```

3. The Mental Model

There are three layers, and they are kept strictly separate. Understanding the split explains most design decisions in the code.

Layer 1 - Description (what the robot IS)

A set of xacro files that declare the robot: link geometry, joints, sensors, the hardware interfaces `ros2_control` will drive, and the Gazebo plugins. Nothing here knows about the mission. Every dimension is a named constant, declared once.

Layer 2 - Simulation assets (where the robot LIVES)

World files (room, walls, shelf, objects at their starting poses), object models, and one launch file that starts every process in a deterministic order. Nothing here knows about kinematics.

Layer 3 - Behaviour (what the robot DOES)

Pure-Python kinematics and choreography with no ROS imports at all, plus two ROS nodes: one that turns camera frames into detections, and one state machine that runs the mission. Because the kinematics and choreography are ROS-free, the entire motion plan can be validated by a unit test on any machine without launching Gazebo.

The most important consequence of this split

Geometry is declared in the xacro files and mirrored as constants in `ik.py`. A test asserts that every motion the mission can command is reachable within the declared joint limits. If you change a link length in the xacro, you must change it in `ik.py` too - and the test is what will catch you if you forget.

4. Repository Map

bimanual_ws/	
README.md	quick reference: setup, run, tuning
src/	
kobuki_bimanual_description/	WHAT THE ROBOT IS
urdf/robot.urdf.xacro	top level: all constants + assembly
urdf/base.xacro	base body, deck, wheels, casters
urdf/mast_sensors.xacro	twin mast rods, LiDAR, camera
urdf/arm.xacro	one 4-DOF arm + gripper (used twice)
urdf/ros2_control.xacro	12 joint interfaces + tracking gain
urdf/gazebo_plugins.xacro	diff drive, grasp joints, control
config/controllers.yaml	four controller definitions
CMakeLists.txt package.xml	build + manifest
kobuki_bimanual_gazebo/	WHERE THE ROBOT LIVES
worlds/main.sdf	room + shelf + all three objects
worlds/bottle.sdf	room + shelf + bottle only
worlds/bowl.sdf	room + shelf + bowl only
worlds/box.sdf	room + shelf + cracker box only
models/bottle/	model.sdf + model.config
models/bowl/	model.sdf + model.config
models/cracker_box/	model.sdf + model.config
models/shelf/	model.sdf + model.config (static)
launch/sim.launch.py	the single bring-up file
CMakeLists.txt package.xml	build + manifest
kobuki_bimanual_behavior/	WHAT THE ROBOT DOES
kobuki_bimanual_behavior/	
ik.py	analytic FK/IK, no ROS imports
choreography.py	waypoint generators, no ROS imports
arm_commander.py	waypoints -> JointTrajectory msgs
perception_node.py	camera -> /detections (ROS node)
mission_node.py	the mission state machine (ROS node)
__init__.py	empty package marker
config/mission_params.yaml	every tunable number
test/test_ik.py	offline validation suite
setup.py setup.cfg	entry points + install layout
package.xml resource/	manifest + ament marker

Two files you will edit most often

config/mission_params.yaml for behaviour tuning (distances, colour ranges, grasp offsets, speeds), and mission_node.py for logic changes. Geometry changes mean editing robot.urdf.xacro AND ik.py together.

5. Package 1 - kobuki_bimanual_description

Build type ament_cmake. Installs the urdf/ and config/ directories into the share directory so the launch file and controller manager can find them. Contains no executable code.

5.1 urdf/robot.urdf.xacro - the top level

This is the file the launch process feeds to xacro. It does three things: declares every geometric constant as an xacro property, includes the other five xacro files, and calls their macros to assemble the robot. Read this file first - every number in the robot is here.

Constants declared here (all lengths in metres, angles in radians)

Property	Value	Meaning
base_radius	0.1775	Kobuki radius (354 mm diameter)
base_body_z0 / base_body_top	0.020 / 0.125	Bottom and top of the base shell above ground
deck_top_z	0.140	Arm mounting deck: base height + 15 mm standoff
wheel_radius / wheel_sep	0.038 / 0.230	Drive wheel geometry
caster_radius	0.019	Front and rear support spheres
arm_mount_x / arm_mount_y	0.095 / 0.08875	Arm mount offsets from base centre (radius 0.130, separation 0.1775)
shoulder_z	0.180	Shoulder pitch axis height = deck + 40 mm riser
L1 / L2	0.160 / 0.160	Upper arm and forearm lengths
mast_x	-0.100	Mast assembly offset (behind base centre)
lidar_z	0.460	LiDAR height above ground
cam_x / cam_z / cam_pitch	-0.075 / 0.420 / 0.2618	Camera position and 15 degree downward tilt
yaw_lim / pitch_lim	2.618 / 2.0	Joint limits: +/-150 deg yaw, +/-115 deg for each pitch
finger_travel	0.065	Maximum prismatic finger opening

Assembly calls at the bottom of the file

```
<xacro:kobuki_base/>           from base.xacro
<xacro:mast_sensors/>         from mast_sensors.xacro
<xacro:arm_4dof side="left" reflect="1"/> from arm.xacro
<xacro:arm_4dof side="right" reflect="-1"/> same macro, mirrored
<xacro:bimanual_ros2_control/> from ros2_control.xacro
<xacro:bimanual_gazebo/>      from gazebo_plugins.xacro
```

Vertical datum - important

base_link sits at GROUND level directly under the base centre, not at the base body centre. That means every z value in this project reads directly as height above ground, which is why the LiDAR is "0.46" and the shoulder is "0.18". Keep this convention if you add links.

5.2 urdf/base.xacro - body, wheels, deck

- Defines two reusable inertia helper macros, cyl_inertial and box_inertial, which compute inertia tensors from mass and dimensions so no hand-computed numbers are needed.
- base_footprint (ground projection) is the root link; base_link is fixed to it at the same position. The base shell is a cylinder with a cosmetic red bumper ring, mass 3.2 kg (representing battery and electronics; a heavier base resists tipping under arm loads).

- `deck_link` is a thin disc fixed at 0.140 m - the plate the arms bolt to. On the real robot this is a hexagon; here a cylinder of the same radius.
- Two continuous drive wheel joints, `left_wheel_joint` and `right_wheel_joint`, at ± 0.115 m in y, with friction coefficients set to 1.0. These are the joints the DiffDrive plugin drives.
- Two caster spheres at $x = \pm 0.15$ m with friction set to ZERO, so they support the chassis without resisting turns.

5.3 urdf/mast_sensors.xacro - the sensor mast

Implements the twin-rod mast from the design blueprint: two vertical posts, each 0.14 m from the base centre on the rear half and symmetric about the centreline, joined by a crossbar that carries the sensors. All of it is one link (`mast_link`) with multiple visuals, because the mast is rigid - there is no reason to pay for extra links.

The two sensors, declared as Gazebo sensor tags

LiDAR	type <code>gpu_lidar</code> on <code>lidar_link</code> at $z = 0.46$. 360 samples over a full circle, range 0.12 to 8.0 m, 10 Hz, publishing on the Gazebo topic "scan".
Camera	type <code>camera</code> on <code>camera_link</code> at $z = 0.42$, pitched 15 degrees down. 640x480 RGB, 69 degree horizontal FOV, 15 Hz, publishing on "camera" with intrinsics on "camera_info".

Why the camera does object detection and the LiDAR does not

A horizontal 2D scan plane at any practical mast height cannot intersect a 5-10 cm tall object lying on the floor. The LiDAR is therefore used for room structure, the forward safety guard, and finding the shelf board. Object detection and ranging are done from the camera image alone.

5.4 urdf/arm.xacro - one arm, instantiated twice

A single macro, arm_4dof, parameterised by side ("left"/"right") and reflect (+1/-1) so the same code generates both arms mirrored about the centreline. The kinematic chain is:

```
base_link
--J1 (revolute, yaw about z) --> <side>_shoulder_link (40 mm riser)
--J2 (revolute, pitch about y)--> <side>_upper_arm_link (L1 = 160 mm)
--J3 (revolute, pitch about y)--> <side>_forearm_link (L2 = 160 mm)
--J4 (revolute, pitch about y)--> <side>_palm_link
    + <side>_finger_l_link (prismatic, +y)
    + <side>_finger_r_link (prismatic, -y)
```

Conventions you must not break

- J1 is a yaw axis passing vertically through the shoulder, which is what makes the inverse kinematics decouple exactly into a yaw plus a planar three-pitch problem.
- Positive pitch rotates the arm DOWNWARD (right-hand rule about +y). ik.py depends on this sign convention.
- Finger joints are prismatic and POSITIVE MEANS OPEN - each finger moves outward by its joint value. Travel is 8 to 65 mm.
- The palm is 145 mm wide so that it spans the full finger travel; if you shorten it, fully opened fingers will appear detached from the arm.
- Finger pads carry friction coefficients of 1.8 (high grip), and each finger is 75 mm long - long enough to straddle the tallest object cross-section without the palm striking it.

Joint efforts and velocities

J1-J3 effort 6.0 N m, velocity 3.0 rad/s

J4 (wrist) effort 4.0 N m, velocity 3.0 rad/s

Fingers effort 30 N, velocity 0.2 m/s

Rationale A 500 g payload at full 0.32 m extension needs about 1.6 N m at the shoulder, so 6 N m leaves margin for link weight - consistent with mid-tier Dynamixel XM-class servos.

5.5 urdf/ros2_control.xacro - the hardware interface

Declares the ros2_control system that gz_ros2_control implements. All twelve controllable joints (eight arm joints and four finger joints) are listed with a position command interface and position plus velocity state interfaces. Finger joints get an initial value of 0.060 (open).

The single most important number in the project

This file sets position_proportional_gain to 15.0. The gz_ros2_control default is 0.1, which produces roughly a ten-second lag: the arms never reach their commanded poses, sag under gravity and drag on the floor while driving. If arm behaviour ever looks sluggish or droopy again, check this value first.

5.6 urdf/gazebo_plugins.xacro - three plugins

DiffDrive Drives left_wheel_joint and right_wheel_joint from /cmd_vel, publishes /odom and a TF stream on /tf_gz at 30 Hz, with acceleration limits.

DetachableJoint x3 One per object. Each can create or destroy a rigid fixed joint between left_palm_link and that object model's "link", commanded on /grasp/<obj>/attach and /grasp/<obj>/detach, with feedback on /grasp/<obj>/state. This IS the grasp.

gz_ros2_control Hosts the controller manager inside the Gazebo process and loads config/controllers.yaml.

Behaviour you must know about DetachableJoint

These joints exist ATTACHED when the world loads. That is why the first thing mission_node does is send confirmed detach commands for every object. In the single-object worlds, the plugins for the two absent objects log errors at startup and then sit inert - that is expected and harmless.

5.7 config/controllers.yaml - four controllers

Controller	Type	Joints / interface
joint_state_broadcaster	JointStateBroadcaster	publishes all 12 joints on /joint_states
left_arm_controller	JointTrajectoryController	left_j1..left_j4, position command, spline interpolation
right_arm_controller	JointTrajectoryController	right_j1..right_j4, position command, spline interpolation
gripper_controller	JointGroupPositionController	the four finger joints, driven by one Float64MultiArray

The manager runs at 200 Hz with use_sim_time true. Trajectory goal time tolerance is set to 0 and non-zero end velocity is allowed, so the controllers accept the multi-waypoint trajectories the mission streams without rejecting them for timing.

6. Package 2 - kobuki_bimanual_gazebo

Build type `ament_cmake`. Installs `worlds/`, `models/` and `launch/`. This package holds the environment and the single launch file that starts everything.

6.1 models/ - four models

Each model directory holds a `model.config` (metadata Gazebo reads to index the model) and a `model.sdf` (the actual geometry, mass, inertia and material). They are referenced from the worlds as `model://<name>`, which resolves because the launch file appends `models/` to `GZ_SIM_RESOURCE_PATH`.

Model	Contents	Notes
bottle	Cylinder r 0.03 x 0.20, mass 0.15 kg, plus a small cap visual	Orange, highly saturated so the colour filter is unambiguous
bowl	Cylinder r 0.04 x 0.10, mass 0.20 kg, with a rim visual	A tall cup rather than a shallow bowl: a shallow 10 cm-wide dish left only 6 mm of finger clearance per side, which is not a workable grasp
cracker_box	Box 0.06 x 0.08 x 0.18, mass 0.25 kg	Green; 8 cm width leaves 16 mm of finger clearance per side
shelf	Static: 0.25 x 0.40 slab with its top surface at 0.12 m, four legs, and a 0.40 x 0.60 m back board	The back board exists so the 0.46 m LiDAR plane can see the shelf. All surfaces are near-zero saturation grey

Palette rule - do not repaint these

The shelf is grey and the arm links are desaturated slate on purpose. Wooden brown shares a hue family with the orange bottle, and saturated blue arms fall inside the bowl colour mask. Either one produces confident false detections. If you add scenery, keep it desaturated.

6.2 worlds/ - four worlds

All four share the same room: a 6 x 6 m floor with four low walls, a sun light, physics at a 1 ms step, and the Physics, UserCommands, SceneBroadcaster and Sensors (ogre2) system plugins. The shelf sits at $x = -1.2$, $y = 0$ with its front face toward $+x$. The robot spawns at the origin facing $+x$.

World	Objects present	Object poses
main.sdf	bottle, bowl, cracker_box	Spread in front of the robot at roughly (1.6, 0), (1.3, -1.0) and (1.2, 1.1) - each lying down and angled so its long axis points back toward the robot start area
bottle.sdf	bottle only	Straight ahead at (1.6, 0), lying
bowl.sdf	bowl only	Straight ahead at (1.6, 0), lying
box.sdf	cracker_box only	Straight ahead at (1.6, 0), lying

"Lying" is expressed as a pitch of 1.5708 rad in the include pose, which tips the object's long axis from vertical to horizontal. The z value in each pose equals the object's resting half-thickness (for example 0.041 for the bowl, whose radius is 0.04) so it starts exactly on the floor rather than dropping into it.

Why the objects are angled toward the robot start area

The grasp choreography assumes the object's long axis is aligned with the approach direction, because the two hands grip its two ends front-and-back. The mission enforces this by always approaching from a central scan hub. If you move objects, keep their axes pointing roughly at the origin.

6.3 launch/sim.launch.py - the bring-up file

One file starts every process, in a guaranteed order. It declares two launch arguments, sets the model search path, and then builds a chain of event handlers so that nothing starts before its prerequisite has finished.

What it declares and starts

world (arg)	default "main"; selects worlds/<world>.sdf
autostart (arg)	default "true"; passed to mission_node so you can load the sim without running the mission
GZ_SIM_RESOURCE_PATH	appended with this package's models/ directory so model:// URIs resolve
gz_sim	includes ros_gz_sim/gz_sim.launch.py with gz_args "-r -v3 <world path>" (-r starts the world running)
robot_state_publisher	robot_description parameter is the OUTPUT OF RUNNING xacro on robot.urdf.xacro at launch time
ros_gz_sim create	spawns the robot from the /robot_description topic at x 0, y 0, z 0.005, named kobuki_bimanual
ros_gz_bridge	one parameter_bridge process carrying clock, scan, camera, camera_info, cmd_vel, odom, TF and the nine grasp topics; remaps /tf_gz to /tf

The startup ordering chain - why it is strictly serial

```
spawn_robot --OnProcessExit--> joint_state_broadcaster spawner
--OnProcessExit--> left_arm_controller spawner
--OnProcessExit--> right_arm_controller spawner
--OnProcessExit--> gripper_controller spawner
--OnProcessExit--> [3 s timer] perception + mission
```

Do not parallelise the spawners

Running the four controller spawners at once makes them contend for the controller manager lock; during development this caused load failures and the mission starting while the right arm controller was still inactive - so one arm silently sat out the grasp. The serial chain plus a readiness gate inside mission_node is what prevents that.

7. Package 3 - kobuki_bimanual_behavior

Build type ament_python. Two console entry points are declared in setup.py: perception_node and mission_node. The other three modules are libraries. ik.py and choreography.py import nothing but the standard library math module, which is what makes them unit testable without ROS.

7.1 ik.py - analytic forward and inverse kinematics

Mirrors the arm geometry from the xacro as module constants: SHOULDER_X 0.095, SHOULDER_Y 0.08875, SHOULDER_Z 0.180, L1 0.160, L2 0.160, LG 0.070 (wrist to grasp point), YAW_LIM 2.618, PITCH_LIM 2.0.

Public functions

shoulder(side)	Returns the shoulder position for side = +1 (left) or -1 (right).
fk(q, side)	Forward kinematics. Given joint vector [q1..q4] returns the grasp point as (x, y, z, pitch) in base_link.
solve(x, y, z, pitch, side)	Inverse kinematics. Returns [q1..q4] or raises IKError.
reachable(x, y, z, pitch, side)	Boolean convenience wrapper around solve.

How solve works

- The target is the GRASP POINT - the spot between the finger pads, 0.070 m beyond the wrist - not the wrist itself.
- q1 (yaw) comes directly from atan2 of the target offset from the shoulder. Because the yaw axis passes vertically through the shoulder, this decouples exactly.
- The wrist position is back-computed from the target by subtracting LG along the requested end-effector direction, reducing the rest to a planar two-link problem in the yawed vertical plane.
- The elbow angle follows from the cosine rule. Both elbow-up and elbow-down branches are computed; solutions violating any joint limit are discarded, and the elbow-up branch is preferred when both are valid.
- If nothing survives, IKError is raised with a descriptive message. The mission catches this rather than commanding a bad pose.

Sign conventions - these must match the xacro exactly

```
q1 yaw about +z, 0 = arm pointing along +x
q2,q3,q4 pitch about +y; POSITIVE PITCH = ARM MOVES DOWN
target "pitch" elevation of the end-effector axis:
0 = horizontal, pointing away from the shoulder
-pi/2 = pointing straight down (this is the grasp approach)
```

7.2 choreography.py - where the arms go, and when

Generates lists of waypoint PAIRS. One pair is ((x,y,z,pitch) for the left hand, (x,y,z,pitch) for the right hand). Both arms work on the robot centreline (y = 0) except during placement, where a lateral offset is applied.

Constants

Constant	Value	Meaning
GRASP_FORWARD	0.30	Object centre distance ahead of the base centre at grasp time
FLIP_X / FLIP_Z	0.35 / 0.20	Object centre position while flipping
HOVER_DZ	0.12	Pre-grasp hover height above the lying object centre
PHI_LIFT	0.35	Radians of pre-rotation applied during the lift phase
CLEAR_DZ	0.08	Vertical clearance move after release

Constant	Value	Meaning
TUCK_LEFT / TUCK_RIGHT	[0.8,-1.5,1.85,1.2] / mirrored	Folded travel pose in JOINT space - hands high at the sides, out of the camera view
STAGE_LEFT / STAGE_RIGHT	[-0.3,-1.0,1.5,0.6] / mirrored	Intermediate pose so tuck-to-grasp interpolation cannot sweep the wrist through the base deck

The generators

_stations(cx, cz, phi, s)	The heart of the module. For an object whose centre is at (cx, 0, cz) with its axis at angle phi from horizontal, it returns the two hand grasp points at +/-s along that axis, with the hand pitch set to phi - pi/2. Everything else is built from this.
grasp_waypoints	Two pairs: hover above the lying object, then descend to grasp height. Hands point straight down.
flip_waypoints	Twelve pairs in two phases. Phase A (4 pairs) raises the object centre from the grasp spot to the hover point while pre-rotating only to PHI_LIFT. Phase B (8 pairs) rotates phi the rest of the way to pi/2 about the now-fixed centre.
place_waypoints	One pair: object upright (phi = pi/2) lowered to the shelf height, shifted laterally by y_off.
clear_waypoints	One pair: same lateral position, CLEAR_DZ higher, so the hands lift straight off the placed object.

Why the flip is two-phased

A single continuous arc from the floor to upright drove the lower hand into the region too close to its own shoulder, where the elbow angle exceeds the joint limit. Lifting first and rotating afterwards at a farther-out hover point keeps both wrists inside the reachable band the whole way. The offline test proves this for every waypoint.

7.3 arm_commander.py - waypoints to ROS messages

A thin translation layer. It owns the joint name lists (LEFT_JOINTS, RIGHT_JOINTS) and converts joint vectors or cartesian waypoint pairs into trajectory_msgs/JointTrajectory.

joint_traj(names, q_list, dt, t0)	Builds a trajectory straight from joint vectors, spaced dt seconds apart starting at t0. Used for tuck and staging poses.
pair_trajs(waypoint_pairs, dt, t0)	Runs ik.solve on every waypoint for the correct side and returns a matched (left, right) trajectory pair. Raises IKError if any waypoint is unreachable - the mission never sends a partially valid motion.
final_positions(traj)	Dict of joint name to final commanded position; used to test for arrival.
total_time(traj)	Duration of the last point, used to size the wait.

Because both trajectories in a pair share identical timestamps, the two joint trajectory controllers execute them in lockstep, which is what keeps the rigid two-hand constraint from being violated during the flip.

7.4 perception_node.py - camera to detections

A single ROS node. Subscribes to /camera (sensor_msgs/Image) and /camera_info, publishes /detections (std_msgs/String containing JSON). It runs on every camera frame, roughly 15 Hz.

Parameters it reads (from mission_params.yaml)

cam_height / cam_pitch / cam_x	0.42 m, 0.2618 rad, -0.075 m - the camera pose used for geometry
min_area	120 px, the smallest blob accepted as a detection
objects	list of object names to look for
<object>.hsv_lo / hsv_hi	per-object HSV threshold bounds in OpenCV ranges (H 0-179, S and V 0-255)

The per-frame algorithm, in order

- on_image converts the raw buffer to a numpy array and then to HSV (handling both rgb8 and bgr8 encodings).
- For each object, cv2.inRange builds a colour mask, findContours extracts blobs, and the largest by area is kept if it exceeds min_area.
- From that blob's bounding rectangle it takes the horizontal midpoint (for bearing) and the BOTTOM pixel row (for range).
- ground_range(v_bottom) computes the depression angle as $\text{cam_pitch} + \text{atan2}(v - \text{cy}, \text{fy})$, then returns $\text{cam_height} / \tan(\text{depression})$ - the horizontal distance from the camera to the point where the object meets the floor. Returns None if the depression is too shallow to intersect the floor.
- range_base adds cam_x, converting a camera-relative distance into a distance from the base centre, which is the frame the mission reasons in.
- bearing is $-\text{atan2}(u_{\text{mid}} - \text{cx}, \text{fx})$: positive means the object is to the robot's left, matching the sign of angular velocity.
- est_h is $\text{blob_height_px} * \text{range} / \text{fy}$ - the apparent PHYSICAL height of the object in metres. A lying bottle measures about 6 cm, an upright one about 20 cm, which is how the mission tells them apart.
- All results are packed into one JSON dict and published, so the mission has a single self-describing topic to read.

Example /detections payload

```
{"bottle": {"found": true, "bearing": 0.021, "range_base": 1.43,
            "est_h": 0.062, "area": 812},
 "bowl": {"found": false},
 "cracker_box": {"found": true, "bearing": -0.55, ...}}
```

Intrinsics

Focal lengths default to values derived from the modelled 640 px width and 69 degree horizontal FOV, and are overwritten by `on_info` as soon as a real `/camera_info` message arrives. So the geometry is correct even if you change the camera resolution in the xacro.

7.5 mission_node.py - the mission state machine

The largest file in the project and the one that decides everything. It is a ROS node whose callbacks keep a small world model up to date, plus a worker THREAD that runs the mission as straight-line procedural code. This is deliberate: the mission reads top to bottom like a recipe instead of being scattered across callbacks.

Structure of the file

norm_ang(a)	Module helper; wraps an angle into -pi..pi.
DEFAULT_OBJECTS	Fallback per-object geometry, overridden by the parameter file.
ARM_JOINTS	The 12 joint names used by the readiness gate.
MissionNode.__init__	Reads parameters, creates publishers and subscriptions, then starts the worker thread.
Callbacks	_on_det, _on_odom, _on_js, _on_scan, plus one lambda subscription per object for grasp state.
Low-level helpers	log, cmd, stop, fresh, grippers, set_grasp, send_pair, move_arms, tuck, wait_odom.
Base motion	rotate_by, rotate_until_found, center_on, drive_odom, goto.
Docking	_board_cluster, _dock_servo, dock_to_shelf.
Task level	approach, pick_flip_place.
Top level	_safe_run wrapping run().

The world model kept by callbacks

Field	Filled by	Contents
self.det / det_stamp	_on_det	Latest parsed /detections dict and its arrival time
self.odom	_on_odom	(x, y, yaw) - yaw extracted from the quaternion
self.joints	_on_js	joint name to position, all 12 joints
self.scan	_on_scan	The full LaserScan message, kept for docking
self.front_min	_on_scan	Closest valid return in a ~30 degree forward sector - the safety guard
self.grasp_state	per-object lambda	obj name to "attached" or "detached", straight from the plugin

fresh(name) - the staleness guard

Every read of a detection goes through fresh(), which returns None if the last /detections message is older than 0.6 s or if that object was not found in it. This is why a dead perception node makes the robot stop looking rather than act on stale data.

Key helpers explained

grippers(val, settle)	Publishes one Float64MultiArray of four identical finger positions, then sleeps to let them move.
set_grasp(name, attach)	THE CLOSED-LOOP GRASP. Publishes the attach or detach command up to six times and waits for the plugin's state topic to report the wanted state. A single dropped message would otherwise leave an object silently welded to the palm, which poisons everything downstream.
send_pair(traj_l, traj_r)	Publishes both arm trajectories, then polls /joint_states until every commanded joint is within 0.06 rad of its final target, or a deadline of the trajectory duration plus 3.5 s expires. Returns True only on convergence, which is how the mission detects a blocked arm.
move_arms(pairs, dt, t0)	Convenience: pair_trajs then send_pair.
tuck()	Sends both arms to the folded travel pose.
rotate_by(delta, note_seen, seen)	Integrates yaw from odometry and spins until delta is covered. When note_seen is given it records every object detected during the turn - this is the survey.

rotate_until_found(name)	Spins looking for one specific object; requires FOUR consecutive detections before accepting, which rejects single-frame colour noise.
center_on(name, tol)	Proportional control on bearing until the object is centred within tol radians.
drive_odom(dist, guard)	Straight drive by odometry, negative for reverse. With guard=True it aborts if front_min drops below front_guard.
goto(gx, gy, gyaw, slow)	Turn-drive-turn to an odometry pose, with a re-aim if heading error exceeds 0.8 rad. slow=True caps speed at 0.14 m/s and turn rate at 0.35 rad/s, used whenever an object is being carried.

Docking internals

`_board_cluster()`

Groups contiguous scan returns that lie within ± 0.9 rad of straight ahead and closer than 1.6 m. A candidate is accepted only if it has at least 3 beams, its PHYSICAL width (computed as $2 * \text{range} * \tan(\text{half angular span})$) falls in 0.15-0.70 m, and it stands out in DEPTH - the beams just outside both edges must be missing or at least 0.4 m farther away. The width test alone is not enough: a wall clipped by the range limit can fake a narrow cluster, and the depth test is what rejects it.

`_dock_servo()`

Servos onto an accepted cluster: forward or backward on range error toward shelf_stop_range, turning on bearing error, converging when the range is within 2 cm and the bearing within 0.04 rad. If no cluster is visible it performs a BOUNDED sweep of about ± 40 degrees rather than a free spin, so it can never lock onto something behind the robot. Returns "docked", "lost" or "notfound".

`dock_to_shelf()`

Two attempts. A successful dock is cross-checked against odometry and rejected as a phantom if the two disagree by more than 0.6 m. Between attempts it reverses 0.35 m before turning (the arms plus payload sweep about 0.45 m and would otherwise snag the shelf board) and re-approaches. If both attempts fail it returns False and the placement happens at the odometry pose - a bounded fallback rather than wherever a servo happened to stop.

approach(name) - the two-stage drive

- Stage 1, visual servo: while the object is visible and its range_base exceeds approach_stop_range (0.55 m), drive forward at v_lin when the bearing is small and steer proportionally. The forward LiDAR guard aborts the approach if something is closer than front_guard.
- Stage 2, blind creep: below 0.55 m the object leaves the camera field of view. Before losing it, the robot re-centres tightly (tolerance 0.015 rad) and refreshes its range estimate, then computes creep = (object centre distance) - grasp_forward and drives exactly that far on odometry with the guard disabled.
- The object centre distance is range_base + half_len, because perception measures the near floor contact edge while the grasp needs the centre.

pick_flip_place(name) - the manipulation sequence in order

<code>grasp_z = lying_z + grasp_z_off</code>	hands grip above the centreline
<code>place_z = z_place + grasp_z_off</code>	same offset honoured at release
1 <code>grippers(open_pos)</code>	open both hands
2 <code>send STAGE_LEFT / STAGE_RIGHT</code>	raise clear of the deck
3 <code>hover LEFT, wait, hover RIGHT</code>	staggered: arms never cross
4 <code>move_arms(grasp_waypoints)</code>	hover then vertical descent
-> if it fails: reopen, re-hover, retry once	
-> if it fails again: open, tuck, back off, return False	
5 <code>grippers(close)</code>	light-touch fit on the object
6 <code>set_grasp(name, True)</code>	CONFIRMED rigid attach
7 <code>grippers(close + 0.012)</code>	relieve fingers 12 mm
8 <code>move_arms(flip_waypoints)</code>	lift then rotate to upright
9 <code>goto(shelf_approach, slow=True)</code>	loaded transit, limited speed
10 <code>dock_to_shelf()</code>	LiDAR-referenced final pose
11 <code>move_arms(place_waypoints(y_off))</code>	lower into this object's slot
12 <code>set_grasp(name, False)</code>	CONFIRMED release
13 <code>grippers(open_pos)</code>	open hands
14 <code>move_arms(clear_waypoints(y_off))</code>	lift straight off the object
15 <code>drive_odom(-backup_dist)</code>	reverse clear of the shelf
16 <code>tuck()</code>	fold arms, ready for the next

Step 7 is not cosmetic

Once the rigid joint carries the object, closed fingers are a second controller fighting the first. Leaving them clamped made the object visibly shift and appear to pass through the gripper during the flip. Backing them off 12 mm removes the conflict while still looking like a grasp.

run() - the top level

```
1 wait for odometry          (_on_odom has fired)
2 wait for ALL 12 joints to report  readiness gate
3 confirmed detach of every object  joints start ATTACHED
4 open grippers, tuck arms        safe initial state
5 survey: rotate_by(2*pi + 0.3)    record every object seen
6 for each object, inside try/except:
    a return to the scan hub        repeatable approach angle
    b rotate_until_found            4 consecutive detections
    (fallback: second vantage point at x = 0.45)
    c center_on                    square up to it
    d skip if already upright        est_h >= 0.7 * upright_h
    e approach                     servo + blind creep
    f pick_flip_place               the whole manipulation
    on ANY exception: release, open, tuck, reverse, next object
7 log MISSION COMPLETE
```

`_safe_run` wraps `run()` so an unexpected error is logged and the base is stopped rather than leaving the robot driving. The per-object try/except inside the loop is what guarantees one bad object cannot end the mission.

7.6 config/mission_params.yaml - every tunable number

Two parameter blocks, one per node. This file is the first place to look when behaviour is wrong but the code is right.

perception_node parameters

cam_height, cam_pitch, cam_x	Camera pose; must match the xacro
min_area	Minimum blob area in pixels
bottle.hsv_lo / hsv_hi	H 5-22 with a saturation floor of 170 - the high floor is what keeps wood and grey tones out of the orange mask
bowl.hsv_lo / hsv_hi	H 100-135 with a saturation floor of 130 - high enough to exclude the desaturated arm links, low enough that lighting washout does not hide the bowl
cracker_box.hsv_lo / hsv_hi	H 40-80, saturation floor 100

mission_node parameters - global

objects	Which objects to hunt, IN PRIORITY ORDER
grasp_forward	0.30 m - object centre distance ahead of the base at grasp
approach_stop_range	0.55 m - where the visual servo hands over to the blind creep
shelf_approach	[-0.83, 0.0, pi] - the odometry pose used to get near the shelf before docking
shelf_stop_range	0.594 m - the LiDAR-measured board range that defines "docked"
scan_hub	[0.0, 0.0] - the central point the robot returns to before hunting each object
backup_dist	0.6 m reverse after placing
open_pos	0.062 m per finger when open
v_lin, w_rot, scan_w	0.22 m/s, 0.5 rad/s, 0.4 rad/s search spin
front_guard	0.35 m forward LiDAR safety stop

mission_node parameters - per object

Key	Meaning	bottle	bowl	box
half_len	Half the lying long dimension	0.10	0.05	0.09
lying_z	Centre height when lying	0.031	0.041	0.031
s	Grasp station offset along the axis	0.050	0.035	0.050
close	Finger position when closed on it	0.037	0.047	0.047
grasp_z_off	Grip this far above the centreline	0.010	0.010	0.010
z_place	Object centre height at release	0.225	0.175	0.215
place_y_off	Lateral slot on the shelf	0.00	+0.10	-0.10
upright_h	True height when standing	0.20	0.10	0.18
check_upright	Run the already-standing test?	true	false	true

Two subtleties in this table

place_y_off gives each object its own slot on the 0.40 m slab so a newly placed object cannot knock over an earlier one. check_upright is false for the bowl because a cylinder of similar height and width looks the same lying or standing, so the height test cannot distinguish its pose.

7.7 test/test_ik.py - the offline validation suite

Runs with plain python3, no ROS, no Gazebo: "python3 test/test_ik.py" from the package directory. Four suites.

Test	What it asserts
test_roundtrip_grid	Over a grid of workspace poses for both arms, every reachable target solves and forward kinematics reproduces it to within 1e-9 m and rad

Test	What it asserts
test_unreachable_raises	A target a metre away raises IKError instead of silently returning a bad solution
test_mission_choreography_reachable	THE IMPORTANT ONE. For all three objects it regenerates every grasp, flip, place and clearance waypoint exactly as the mission would - including the grasp height offsets and lateral placement offsets - and asserts each one is reachable by the correct arm within joint limits
test_tuck_pose_within_limits	Tuck and staging poses are inside joint limits and place the hands above the deck and inside the reach envelope

Run this before every commit

This suite has already caught three real geometric defects: an elbow limit that was too tight, a flip arc that passed through an unreachable region, and two poses that overreached. It converts "the arm might not reach" from a runtime mystery into a build-time failure.

8. How the Files Interact

Build-time and launch-time flow

```
robot.urdf.xacro (includes base, mast_sensors, arm,
                  ros2_control, gazebo_plugins)
    | xacro is executed BY THE LAUNCH FILE at startup
    v
robot_description (a big URDF string on a ROS topic)
    |
    v
robot_state_publisher (publishes /tf)    ros_gz_sim create --> robot in Gazebo
                                         |
                                         v
                                         plugins declared inside the URDF start:
                                         DiffDrive, 3x DetachableJoint,
                                         gz_ros2_control (reads controllers.yaml)
```

Run-time data flow

```
Gazebo --/camera, /camera_info--> perception_node
                                     |
                                     /detections (JSON)
                                     v
Gazebo --/scan, /odom, /joint_states--> mission_node
Gazebo --/grasp/<obj>/state-----> mission_node
                                     |
                                     /cmd_vel -----> DiffDrive (base moves)
                                     /<side>_arm_controller/joint_trajectory
                                     -----> JTC -> joints move
                                     /gripper_controller/commands
                                     -----> fingers move
                                     /grasp/<obj>/attach | detach --> DetachableJoint

(every Gazebo <=> ROS hop above passes through ros_gz_bridge)
```

Module-level dependencies inside the behaviour package

```
mission_node.py
    imports arm_commander (to build JointTrajectory messages)
    imports choreography (to get cartesian waypoints)
    imports ik (for single-pose solves + IKError)

arm_commander.py imports ik
choreography.py imports nothing but math <-- ROS-free
ik.py imports nothing but math <-- ROS-free
perception_node.py standalone (cv2, numpy, rclpy)
test/test_ik.py imports ik + choreography only
```

The dependency shape is the point

All geometry knowledge flows one way: xacro constants are mirrored in ik.py, choreography builds on ik's frame conventions, arm_commander translates, and mission_node orchestrates. Nothing below mission_node knows about ROS, which is exactly why the test suite can validate the whole motion plan offline.

9. Complete Interface Reference

Topics

Topic	Type	Published by	Consumed by
/clock	roscpp_msgs/Clock	Gazebo (bridged)	every node (use_sim_time)
/scan	sensor_msgs/LaserScan	LiDAR sensor (bridged)	mission_node: _on_scan
/camera	sensor_msgs/Image	camera sensor (bridged)	perception_node: on_image
/camera_info	sensor_msgs/CameraInfo	camera (bridged)	perception_node: on_info
/odom	nav_msgs/Odometry	DiffDrive (bridged)	mission_node: _on_odom
/tf	tf2_msgs/TFMessage	robot_state_publisher + DiffDrive (remapped from /tf_gz)	visualisation
/robot_description	std_msgs/String	robot_state_publisher	controller_manager, create
/joint_states	sensor_msgs/JointState	joint_state_broadcaster	mission_node: _on_js
/detections	std_msgs/String (JSON)	perception_node	mission_node: _on_det
/cmd_vel	geometry_msgs/Twist	mission_node: cmd()	DiffDrive (bridged)
/left_arm_controller/joint_trajectory	trajectory_msgs/JointTrajectory	mission_node	left JTC
/right_arm_controller/joint_trajectory	trajectory_msgs/JointTrajectory	mission_node	right JTC
/gripper_controller/commands	std_msgs/Float64MultiArray	mission_node: grippers()	gripper controller
/grasp/<obj>/attach	std_msgs/Empty	mission_node: set_grasp	DetachableJoint (bridged)
/grasp/<obj>/detach	std_msgs/Empty	mission_node: set_grasp	DetachableJoint (bridged)
/grasp/<obj>/state	std_msgs/String	DetachableJoint (bridged)	mission_node: grasp_state

Joint names - the full set of 12

```
left_j1 left_j2 left_j3 left_j4      (yaw, pitch, pitch, pitch)
right_j1 right_j2 right_j3 right_j4
left_finger_l_joint left_finger_r_joint (prismatic, + = open)
right_finger_l_joint right_finger_r_joint
```

Launch arguments

world main | bottle | bowl | box (default main)
autostart true | false (default true) - false loads the sim without running the mission

10. Frames and Geometry Reference

Frames

base_footprint Root link, ground projection of the base centre.
base_link Fixed to base_footprint at the SAME point, i.e. at GROUND level under the base centre. All z values therefore read as height above ground.
odom Published by DiffDrive. Because the robot spawns at the world origin, the odom frame coincides with the world frame at startup - which is why shelf_approach can be written as a plain odometry coordinate.
IK frame ik.py works entirely in base_link: x forward, y left, z up.

Vertical stack, all above ground

```
0.000 ground / base_link origin
0.019 caster sphere centres
0.038 wheel axle height
0.125 top of the base shell
0.140 arm mounting deck <-- arms bolt here
0.180 shoulder pitch axis (J2) = deck + 40 mm riser
0.420 camera
0.460 LiDAR scan plane

shelf: 0.120 slab top surface
       0.600 top of the back board (the LiDAR docking target)
```

Horizontal layout, in base_link

```
arm mounts      x = +0.095, y = +/-0.08875 (radius 0.130)
mast + sensors  x = -0.100 (rods at y = +/-0.099, radius 0.140)
camera          x = -0.075, pitched 15 deg down
grasp stand-off object centre at x = +0.30 (grasp_forward)
flip / place    object centre at x = +0.35 (FLIP_X)
reach           0.32 m geometric max, 0.20-0.25 m comfortable
```

11. The Full Walkthrough

This is the section to read. It follows one complete run from the moment you press Enter to the MISSION COMPLETE line, and every step names the file and function that causes it. Where a log line appears in the terminal, it is quoted so you can match the narrative to what you see.

How to follow along

Run the sim, keep the terminal visible, and read this section alongside it. The log lines quoted below are the actual strings the nodes print, in the order they print them.

Stage A - Launch and bring-up (about 15 seconds)

A1 The launch file is evaluated

sim.launch.py: generate_launch_description() runs. It reads the world and autostart arguments, appends this package's models/ directory to GZ_SIM_RESOURCE_PATH (so model:// URIs in the worlds resolve), and builds the path worlds/<world>.sdf.

A2 Gazebo starts and loads the world

The ros_gz_sim gz_sim.launch.py include is started with gz_args "-r -v3 <world path>". The -r flag means the world begins stepping immediately. Gazebo loads physics (1 ms step), the ground plane, four walls, the sun, the shelf at x = -1.2, and the object includes at their lying poses.

```
[gazebo-1] [Msg] Loading SDF world file[.../worlds/main.sdf].  
[gazebo-1] [Msg] Gazebo Sim Server v8.15.0
```

A3 The robot description is generated and published

robot_state_publisher is started with its robot_description parameter set to Command(['xacro ', robot.urdf.xacro]) - so xacro is executed at this moment, expanding all six xacro files into one URDF string. The node publishes it on /robot_description and begins publishing /tf.

A4 The robot is spawned

ros_gz_sim create reads /robot_description and injects the robot into the running world at (0, 0, 0.005) as model "kobuki_bimanual", then exits. The plugins declared inside the URDF now come alive: DiffDrive subscribes to /cmd_vel, the three DetachableJoint plugins create their joints (ATTACHED), and gz_ros2_control starts a controller manager inside the Gazebo process, reading controllers.yaml.

```
[create-3] [INFO] Entity creation successful.  
[gazebo-1] [INFO] [gz_ros_control]: Loading controller_manager  
[gazebo-1] [INFO] The position_proportional_gain has been set to: 15
```

A5 The bridge opens

One parameter_bridge process translates every topic listed in the launch file between Gazebo transport and ROS 2, and remaps /tf_gz to /tf. Until this is up, no ROS node can see a sensor.

A6 Controllers activate one at a time

Four spawner processes run in a strict chain, each triggered by the previous one exiting (OnProcessExit). Order: joint_state_broadcaster, then left_arm_controller, then right_arm_controller, then gripper_controller.

```
[spawner-5] Configured and activated joint_state_broadcaster
[spawner-6] Configured and activated left_arm_controller
[spawner-7] Configured and activated right_arm_controller
[spawner-8] Configured and activated gripper_controller
```

A7 The behaviour nodes start

When the gripper spawner exits, a 3 second TimerAction fires and both perception_node and mission_node are started with mission_params.yaml loaded and use_sim_time true.

```
[perception_node] perception up, objects: [bottle, bowl, cracker_box]
[mission_node]    mission: waiting for sim interfaces...
```

Stage B - Mission initialisation

B1 perception_node begins publishing immediately

Its /camera subscription fires at roughly 15 Hz. Each frame runs on_image: HSV conversion, one cv2.inRange mask per object, largest contour, bounding rectangle, ground_range() on the bottom pixel row, bearing from the horizontal offset, est_h from the blob height, then one JSON message on /detections. It does this continuously for the whole run, independent of what the mission is doing.

B2 mission_node waits for the world to be real

run() first calls wait_odom(), which blocks until _on_odom has stored a pose. It then blocks until all twelve names in ARM_JOINTS appear in self.joints, which are filled by _on_js from /joint_states. This is the readiness gate: it guarantees every controller is genuinely live before any command is sent.

```
[mission_node] mission: all 12 joints reporting, controllers ready
```

B3 Startup safety: release, open, tuck

Because the DetachableJoint plugins load ATTACHED, the mission calls set_grasp(name, False) for every object. That helper publishes an Empty on /grasp/<obj>/detach and waits for "detached" to arrive on /grasp/<obj>/state, retrying up to six times. Then grippers(0.062) opens all four fingers, and tuck() folds both arms to TUCK_LEFT / TUCK_RIGHT via arm_commander.joint_traj and send_pair.

Stage C - SEARCH: the survey rotation

C1 One full turn on the spot

rotate_by($2\pi + 0.3$, note_seen=self.objects, seen=seen) is called. Inside, it publishes Twist(angular.z = scan_w = 0.4) every 50 ms and integrates yaw differences from self.odom until the commanded angle is covered. The extra 0.3 rad is overlap so the sweep closes on itself.

C2 Recording what it sees while turning

On every iteration it calls fresh(name) for each object. fresh() reads the cached /detections dict, rejects it if older than 0.6 s, and returns the entry only if that object was found. Any name that appears goes into the seen set. This is why the robot knows what is in the room before it moves toward anything - and why absent objects in the single-object worlds are skipped instantly instead of timing out.

```
[mission_node] mission: survey rotation
[mission_node] mission: survey saw [bottle, bowl, cracker_box]
```

If you see it spin more than once here

That would mean detections are unstable. The survey is exactly one rotation; the shorter spins you see later are rotate_until_found() reacquiring a specific object before its pickup.

Stage D - Per-object cycle, target acquisition

The loop now runs once per object, in the priority order given by the objects parameter. Everything from here to the end of Stage G is wrapped in a try/except so a failure moves to the next object instead of ending the mission.

D1 Return to the scan hub

If the robot is more than 0.15 m from scan_hub (0, 0), goto() drives it back. This matters because the grasp choreography assumes the object's long axis is aligned with the approach direction; always starting from the same central point makes that true for every object.

```
[mission_node] [bowl] returning to scan hub
[mission_node] goto arrived: odom=(-0.035, -0.006, yaw -0.05)
```

D2 Reacquire this specific object

rotate_until_found(name) spins at scan_w and requires FOUR consecutive frames in which fresh(name) returns a detection before it accepts and stops. If 40 seconds pass without that, the robot drives to a second vantage point at (0.45, 0) and tries once more before giving up on the object.

D3 Square up

center_on(name, tol=0.03) applies proportional control on the reported bearing, clamped to 0.4 rad/s, until the object is centred within 0.03 rad.

D4 Skip objects that are already standing

If the object's check_upright flag is true and its est_h is at least 70 percent of its upright_h, it is already vertical and is skipped. A lying bottle reports about 0.06 m and an upright one about 0.20 m, so the test is decisive. The bowl has check_upright false because a cylinder that is as tall as it is wide looks identical either way.

D5 APPROACH - stage 1, visual servo

approach(name) loops: while the object is visible and its range_base is greater than approach_stop_range (0.55 m), it drives at v_lin when the bearing is within 0.25 rad and steers at 1.2 x bearing (clamped). Each iteration also checks self.front_min, maintained by _on_scan from the forward LiDAR sector; if anything is closer than front_guard (0.35 m) the approach aborts rather than pushing into it.

D6 APPROACH - stage 2, the blind creep

At 0.55 m the object is about to leave the camera view. While it is still visible the robot re-centres tightly (tolerance 0.015 rad) and refreshes its estimate, computing the object centre as $\text{range_base} + \text{half_len}$. It then drives exactly $(\text{centre} - \text{grasp_forward})$ metres on odometry with `drive_odom(guard=False)`, which parks the object centre at 0.30 m ahead of the base centre - the pose the grasp choreography assumes.

```
[mission_node] [bowl] approaching  
[mission_node] creeping 0.294 m by odometry
```

Stage E - IK GRASP

E1 Open, then raise to the staging pose

`grippers(open_pos)` opens all four fingers. Then a joint-space trajectory sends both arms to `STAGE_LEFT` / `STAGE_RIGHT`. This exists because the joint trajectory controller interpolates in `JOINT` space: going straight from the folded tuck pose to a low forward grasp pose swings the wrist down through the base deck on the way. The staging pose routes the motion up and over instead.

E2 Deploy to hover, one arm at a time

Both hover stations sit on the centreline, so moving both arms at once makes them cross paths mid-swing. The mission solves the left hover pose with `ik.solve(..., +1)`, publishes only the left trajectory, waits three seconds, then does the right with `ik.solve(..., -1)`. The transient overlap disappears.

E3 Descend and close

`move_arms(choreography.grasp_waypoints(s, grasp_z, grasp_forward))` runs the two-waypoint descent. Internally: `grasp_waypoints` returns hover and grasp pairs with hand pitch `DOWN` ($-\pi/2$) at $x = \text{grasp_forward} \pm s$; `arm_commander.pair_trajs` solves each with `ik.solve` and builds two time-matched `JointTrajectory` messages; `send_pair` publishes both and polls `/joint_states` until every joint is within 0.06 rad of target.

```
[mission_node] [bowl] grasp: descend both grippers
```

If `send_pair` does not converge - which in practice means a finger is touching the object and the arm has stalled - the mission reopens the fingers, lifts back to hover and retries the descent once. A second failure opens the hands, tucks, backs off 0.3 m and abandons the object without ever attaching, so nothing is welded at a wrong pose.

E4 Grip, then create the rigid constraint

`grippers(close)` closes to the object's light-touch fit value. `set_grasp(name, True)` then publishes on `/grasp/<obj>/attach` and waits for the plugin to report "attached" on `/grasp/<obj>/state`, retrying if needed. Only after that confirmation does the mission proceed.

E5 Relieve the fingers

`grippers(close + 0.012)` backs the fingers off 12 mm. From this moment the rigid joint carries the object; leaving the fingers clamped would mean two controllers fighting over the same body, which visibly shoved the object through the gripper during the flip.

```
[mission_node] [bowl] attached; lifting + reorienting to upright
```

Stage F - CO-MANIPULATION and REORIENTATION

F1 Twelve synchronised waypoints

`move_arms(choreography.flip_waypoints(s, grasp_z, grasp_forward))` generates the flip. Phase A is four pairs that raise the object centre from the grasp spot to `(FLIP_X, FLIP_Z) = (0.35, 0.20)` while pre-rotating only to `PHI_LIFT = 0.35` rad. Phase B is eight pairs that rotate the axis angle the rest of the way to $\pi/2$ about that now-fixed centre.

F2 What keeps the grasp valid

Every pair comes from `_stations(cx, cz, phi, s)`, which places the two hands at $\pm s$ along the object axis with hand pitch $\phi - \pi/2$. So the hands are always exactly the object's length apart and always aligned with it. Because both trajectories carry identical timestamps, the two controllers execute them in lockstep and the rigid constraint is never stretched.

Stage G - Transit, docking and PLACEMENT

G1 Loaded transit to the shelf

goto(*shelf_approach, slow=True) drives to the odometry pose (-0.83, 0, pi). slow=True caps speed at 0.14 m/s and turn rate at 0.35 rad/s, because arm reaction torque through a rigidly held payload slips the wheels and corrupts odometry. On arrival the reached pose is logged, which is your direct read-out of accumulated drift.

```
[mission_node] [bowl] upright; navigating to shelf
[mission_node] goto arrived: odom=(-0.792, -0.008, yaw -3.14)
```

G2 Find the shelf with the LiDAR

dock_to_shelf() takes over because the odometry pose alone is not trusted. It calls _dock_servo(), which repeatedly asks _board_cluster() for the shelf back board: contiguous scan returns within +/-0.9 rad and 1.6 m, at least 3 beams, physical width between 0.15 and 0.70 m, and standing out in depth from the beams just outside its edges. When found, the robot servos on range error toward shelf_stop_range (0.594 m) and on bearing error toward zero, converging within 2 cm and 0.04 rad.

```
[mission_node] docked at shelf: board range 0.586 m, bearing 0.035
```

If no board is visible it sweeps a bounded +/-40 degrees rather than spinning freely. A reported dock is cross-checked against odometry and rejected as a phantom if they disagree by more than 0.6 m. On failure the robot reverses 0.35 m (so the arms and payload clear the board before turning), re-approaches and retries once; if that also fails it places at the odometry pose - a bounded fallback.

G3 Lower into this object's own slot

move_arms(place_waypoints(s, place_z, y_off=place_y_off)) lowers the upright object to its release height, shifted laterally by that object's place_y_off so the bottle, bowl and box each occupy a different part of the 0.40 m slab and cannot knock each other over.

```
[mission_node] [bowl] placing on shelf (lateral offset +0.10 m)
```

G4 Release, with confirmation

set_grasp(name, False) destroys the rigid joint and waits for "detached" on the state topic, retrying once more if unconfirmed. This is the step that must not be fire-and-forget: an unconfirmed release leaves the object welded to the palm, and it then gets dragged off the shelf, blocks the camera and snags on the shelf.

G5 Clear, retreat, tuck

grippers(open_pos) opens the hands, move_arms(clear_waypoints(...)) lifts them straight up by CLEAR_DZ (0.08 m) at the same lateral position, drive_odom(-backup_dist) reverses 0.6 m clear of the shelf, and tuck() folds the arms for travel.

```
[mission_node] [bowl] released; backing away
```

Stage H - Loop and termination

Control returns to the top of the per-object loop for the next name. When the list is exhausted the base is stopped and the final line is printed. Any object that failed was released, tucked and backed away from inside the except branch, so the mission always reaches this line rather than hanging.

```
[mission_node] MISSION COMPLETE: all detected fallen objects processed
```

12. Tuning Cookbook

Symptom	What to change
An object is never detected	Widen that object's hsv_lo / hsv_hi in mission_params.yaml, or lower min_area. Lower the SATURATION floor (the second number of hsv_lo) if lighting is washing the colour out
Something that is not the object gets detected	Raise that object's saturation floor, or desaturate the scenery that is matching
Robot stops too far from or too close to the object before grasping	grasp_forward, and approach_stop_range if the handover to the blind creep is happening too early or late
Fingers hit the object or close on nothing	That object's lying_z, s, close and grasp_z_off
Object ends up too high or too low on the shelf	That object's z_place
Two objects collide on the shelf	Their place_y_off values (they must differ by more than the object widths)
Robot parks badly at the shelf	shelf_approach for the coarse pose, shelf_stop_range for the final LiDAR-measured stand-off
Docking never finds the board	The width window and depth contrast threshold in _board_cluster, and max_range / half_fov
Robot drives too fast or too slowly	v_lin, w_rot, scan_w; the loaded transit caps are inside goto(slow=True)
Arms look sluggish or droop	position_proportional_gain in ros2_control.xacro - check this FIRST
Arms move jerkily between waypoints	The dt values passed to move_arms, and see the note below about velocities
"arm trajectory did not fully converge" appears often	The tolerance and extra_wait in send_pair, or the real cause: something is physically blocking the arm

The one improvement worth making first

Trajectory points currently carry positions only, so the controllers interpolate linearly between waypoints. Adding velocities (finite differences between waypoints, zero at the ends) in arm_commander._point makes the controllers use cubic splines instead. It is a small change and it is the single largest visual improvement available.

13. Running the Offline Tests

```
cd src/kobuki_bimanual_behavior
python3 test/test_ik.py

expected output:
PASS test_mission_choreography_reachable
PASS test_roundtrip_grid
PASS test_tuck_pose_within_limits
PASS test_unreachable_raises
```

No ROS, no Gazebo, no build required - it imports only ik.py and choreography.py. Run it after ANY change to link lengths, joint limits, grasp offsets, placement offsets or choreography constants. If you change geometry in the xacro, change the matching constants in ik.py and let this suite confirm the mission is still reachable.

Static checks worth repeating

```
# every xacro / SDF / manifest parses
python3 -c "import xml.etree.ElementTree as ET, glob; [ET.parse(f) for f in glob.glob('src/**/*.xacro', recursive=True)]"

# the URDF actually expands (the real test of the xacro)
xacro src/kobuki_bimanual_description/urdf/robot.urdf.xacro > /tmp/r.urdf

# all python compiles
python3 -m py_compile src/kobuki_bimanual_behavior/kobuki_bimanual_behavior/*.py
```

14. Gotchas for a Newcomer

Things that look like bugs but are not

gz_frame_id SDF warnings	Gazebo warns that gz_frame_id is not a standard SDF element under <sensor>. It is a gz-sim extension and it works. Harmless.
DetachableJoint errors in single-object worlds	The robot always declares three grasp plugins; in a one-object world two of them cannot find their model, log an error and sit inert. Expected.
"Desired controller update period is slower than the gazebo simulation period"	The manager runs at 200 Hz over a 1 ms physics step. Intentional and fine.
Duplicate name messages from DART	Internal renaming of identically named collision shapes across included models. Cosmetic.
The robot spins fully once at the start	That is the survey rotation. Shorter spins later are per-object reacquisition.

Things that will bite you

Geometry lives in two places	The xacro and ik.py. Change one, change the other, then run the test suite.
Grasp joints start ATTACHED	Any code path that skips the startup detach will begin with all three objects welded to the left palm.
The grasp is a weld, not friction	A DetachableJoint fixed joint, per the design blueprint. Do not expect finger friction to hold anything, and do not tighten the fingers onto a welded object.
Self-collision is disabled	Standard for solver stability, but it means the model will happily execute a self-intersecting pose. Correctness comes from the staging poses and the offline pose tests, not from physics.
Everything runs on sim time	use_sim_time is true for every node and /clock is bridged. If you add a node, set it too, or your timing will be wrong.
odom equals world only because of where the robot spawns	The robot starts at the origin, which is why shelf_approach can be a plain odometry coordinate. Move the spawn point and that parameter must move with it.
The mission runs in a worker THREAD	run() blocks with sleeps while rclpy.spin() keeps callbacks alive in the main thread. If you add mission logic, put it in the thread, not in a callback.
Objects must lie with their axis toward the approach	The two hands grip the two ENDS of the object. The scan hub enforces the approach direction; if you reposition objects in a world, keep their axes pointing roughly at the origin.

15. Glossary

xacro	XML macro language. Expands into a URDF. Lets the robot be described with named constants and reusable macros - which is why one arm macro generates both arms.
URDF / SDF	URDF describes the robot; SDF describes Gazebo worlds and models. Gazebo converts the spawned URDF into SDF internally.
ros2_control	The hardware abstraction layer. Declares joint command and state interfaces; controllers are plugins that drive them.
gz_ros2_control	The implementation of ros2_control that talks to Gazebo joints. Runs inside the Gazebo process, which is why controller logs appear with the gazebo prefix.
JTC	JointTrajectoryController. Takes a JointTrajectory (a list of timed joint positions) and interpolates between the points.
spawner	A short-lived process that asks the controller manager to load, configure and activate one controller, then exits. Its exit is what the launch chain waits on.

ros_gz_bridge	Translates messages between Gazebo transport and ROS 2. Nothing crosses between the two worlds without it.
DetachableJoint	A gz-sim system plugin that can create or destroy a fixed joint at run time between two links. This project uses it as the grasp.
FK / IK	Forward kinematics: joints to hand pose. Inverse kinematics: hand pose to joints. Both are implemented analytically in ik.py.
HSV	Hue, Saturation, Value colour space. Hue identifies the colour family, which is far more lighting-stable than raw RGB. OpenCV uses H 0-179, S and V 0-255.
Odometry	Pose estimated by integrating wheel rotation. Drifts, especially under wheel slip - which is exactly why the shelf is located with the LiDAR instead.
Ground-plane back-projection	Recovering distance from a single image by assuming the bottom of an object touches a known floor plane, given a known camera height and tilt.
Scan hub	This project's name for the central point the robot returns to before hunting each object, so every approach direction is repeatable.
Staging pose	An intermediate arm pose used so that joint-space interpolation between two poses cannot pass through the robot itself.

Where to go next in the code

Read `mission_node.run()` top to bottom - it is the whole mission in about forty lines and every other module is called from it. Then read `choreography._stations`, which is the single geometric idea that the entire bimanual manipulation is built on.